NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY
Faculty of Computing and Informatics
School of Computing
Department of Software Engineering

13 Jackson Kaujeua Street
Private Bag 13388
Windhoek
NAMIBIA

T: +264 61 207 2052
F: +264 61 207 9052
E: dse@nust.na
W: www.nust.na

PAP521S – PROGRAMMING IN PRACTICE

PROJECT A: MUNICIPAL FINANCIAL MANAGEMENT SYSTEM

Course: PAP521S – Programming in Practice
Programming Language: ANSI C (C99)
Development Environment: Visual Studio Code + GCC
Version Control: Git & GitHub
Project Type: Group Project
Project A Stage: Foundation System
Group Size: 7 students
Submission: Moodle + GitHub Repository
Due Date: 02 October 2026

1. PROJECT OVERVIEW

Your group has been appointed as a software development team to develop a **Municipal Financial Management System (MFMS)** for a municipality.

The purpose of Project A is to develop the **foundation version** of the system using the programming concepts covered during the first part of the course.

The system must be developed in **C programming language**.

Project A is not expected to be a complete professional municipal ERP system. It is the first functional version of the system that will later be extended in **Project B**.

Your system must demonstrate that your group can apply programming concepts to solve a realistic problem rather than simply reproducing examples from lectures or laboratory exercises.

2. PROJECT A OBJECTIVES

By completing Project, A, your group should demonstrate the ability to:

1. Design a menu-driven C application.
2. Use variables and appropriate data types.
3. Accept and validate user input.
4. Use arithmetic, relational and logical operators.
5. Implement decision-making using `if`, `if-else` and/or `switch`.

6. Use loops appropriately.
7. Use arrays to store and process information.
8. Manipulate strings.
9. Create and use functions.
10. Pass parameters to functions and return values.
11. Organise a program into logical functions/modules.
12. Work collaboratively as a software development team.

These requirements are based on the programming topics covered in Weeks 1–8 of the course. The course outline covers input/output, variables, operators, decisions, loops, arrays, strings and functions during this period.

3. SYSTEM REQUIREMENTS

Your MFMS must contain the following minimum modules.

Module 1 – Main Menu

The system must provide a clear menu similar to:

```
========================================
 MUNICIPAL FINANCIAL MANAGEMENT SYSTEM
========================================

1. Employee Management
2. Budget Management
3. Supplier Management
4. Asset Management
5. Reports
6. Exit

Enter your choice:
```

The design of the menu may be different, but the system must provide clear navigation.

4. EMPLOYEE MANAGEMENT

The system must allow the user to:

- Add an employee.
- Display employees.
- Search for an employee.
- Calculate employee salary information.
- Display relevant employee information.

Each employee should contain appropriate information such as:

- Employee ID
- Name
- Department
- Basic salary

- Housing allowance
- Transport allowance
- Other relevant information

Your group must decide what additional information is appropriate.

The employee module must make meaningful use of:

- Arrays
- Strings
- Functions
- Conditions
- Loops

5. BUDGET MANAGEMENT

The system must allow the user to manage basic municipal budgets.

The system should allow the user to:

- Enter departmental budgets.
- Enter expenditure.
- Calculate remaining budget.
- Determine whether expenditure is within budget.
- Display budget information.
- Identify departments that have exceeded their allocated budget.

Example:

```
Department: Finance
Allocated Budget: N$500000
Expenditure: N$420000
Remaining Budget: N$80000
Status: WITHIN BUDGET
```

Your group should use appropriate calculations and decision-making structures.

6. SUPPLIER MANAGEMENT

The system must allow the municipality to maintain basic supplier information.

At minimum, store:

- Supplier ID
- Supplier name
- Email
- Telephone number
- Town/Location

The system must allow users to:

- Add suppliers.
- Display suppliers.
- Search for suppliers.
- Compare/search supplier information.

The course outline specifically introduces supplier management through the Week 7 practical.

7. ASSET MANAGEMENT

The system must provide a basic asset register.

Examples of municipal assets include:

- Vehicles
- Computers
- Buildings
- Equipment
- Office furniture

The system should store information such as:

- Asset ID
- Asset name
- Asset type
- Purchase value
- Department
- Condition

The system must allow users to display and search asset information.

8. REPORTS

The system must provide basic reports.

At minimum, produce reports such as:

Employee Report

```
Total Employees: 35
Average Salary: N$18,500
Highest Salary: N$42,000
Lowest Salary: N$8,500
```

Budget Report

Show:

- Total allocated budget

- Total expenditure
- Remaining budget
- Departments exceeding budget

Supplier Report

Display the suppliers registered in the system.

Asset Report

Display registered municipal assets.

9. FUNCTIONS

Your system must NOT consist of one very large `main()` function.

You must create appropriate functions.

For example:

```
displayMenu();
addEmployee();
displayEmployees();
searchEmployee();
calculateSalary();
addSupplier();
displaySuppliers();
calculateBudget();
displayReports();
```

The functions listed above are examples only. Your group should determine the most appropriate functions for your design.

The course specifically covers function declarations, parameters, return values, pass-by-value and recursion in Week 8.

10. STRING PROCESSING

Your project must make meaningful use of C string functions.

Examples include:

```
strlen()
strcmp()
strcpy()
strcat()
```

You should use these functions where they are appropriate rather than adding them artificially.

11. DATA VALIDATION

The system should prevent obviously invalid input.

For example:

- Negative salary should not be accepted.
- Negative budget should not be accepted.
- Invalid menu choices should be handled.
- Empty names should be handled appropriately.
- Invalid numerical values should be detected where possible.

12. GROUP RESPONSIBILITIES

Each group will consist of approximately **7 students**.

Each member must have a clearly defined responsibility.

A recommended division is:

Member	Primary Responsibility
Student 1	Employee Management
Student 2	Budget Management
Student 3	Supplier Management
Student 4	Asset Management
Student 5	Reports
Student 6	Functions, integration and validation
Student 7	Testing, documentation and Git coordination

13. IMPORTANT: INDIVIDUAL CONTRIBUTION

Although this is a group project, **every student is individually accountable**.

Every student must:

- Contribute code.
- Understand the complete system.
- Participate in testing.
- Participate in the presentation.
- Maintain evidence of their contribution.
- Have identifiable GitHub activity.

A student must not simply submit a group member's work as their own.

14. GITHUB REQUIREMENT

The group must create a GitHub repository for the project.

The repository should contain:

```
MFMS/
│
├── main.c
├── employees.c
├── employees.h
├── budget.c
├── budget.h
├── suppliers.c
├── suppliers.h
├── assets.c
├── assets.h
├── reports.c
├── reports.h
└── README.md
```

The exact structure may differ depending on your implementation.

Each member must have identifiable contributions.

Do not create the entire project on one computer and make one final GitHub upload.

15. PROJECT A DELIVERABLES

Each group must submit:

1. Source Code

Complete working C source code.

2. GitHub Repository

The URL of the group's GitHub repository.

3. README.md

The README must contain:

- Project title
- Group number
- Group members
- Project description
- System features
- Compilation instructions
- How to run the system
- Individual responsibilities

4. Short Technical Report

Maximum **5 pages**.

The report should contain:

1. Introduction
2. Problem description
3. System objectives
4. System features
5. Program design
6. Challenges encountered
7. Solutions implemented
8. Conclusion

5. Individual Contribution Record

Each student must submit:

- Student name
- Student number
- Assigned responsibility
- Functions/modules developed
- GitHub contribution
- Testing performed

16. PROJECT A DEMONSTRATION

Each group will demonstrate the working system.

The demonstration must show:

1. Main menu
2. Employee management
3. Budget management
4. Supplier management
5. Asset management
6. Searching
7. Reports
8. Input validation
9. Use of functions
10. GitHub repository

The lecturer/assessor may ask any group member to explain or modify part of the program.

17. INDIVIDUAL QUESTIONING

During the demonstration, students should be prepared to answer questions such as:

Explain how your employee search works.

Which function calculates the employee salary?

Why did you use an array here?

What happens if the user enters an invalid menu option?

Where did you use `strcmp()`?

Show me the function that calculates the budget balance.

Explain your contribution to the project.

Show me your GitHub contribution.

A student who cannot explain their claimed contribution may receive a reduced individual mark.

18. PROJECT A ASSESSMENT

Project A will contribute to the **Group Project Progress** component of the course assessment.

Assessment will consider:

Area	Marks
System functionality	20
Application of C programming concepts	20
Functions and program organisation	10
Data handling and validation	10
User interface/menu	10
GitHub/team collaboration	10
Documentation	5
Demonstration	5
Individual understanding/contribution	10
Total	**100**

19. IMPORTANT RULE

Project A is the **foundation** of Project B.

Do not throw away Project A and start a completely unrelated program for Project B.

Project B will extend, improve and refactor Project A.

20. PROJECT A SUCCESS CRITERIA

A successful Project A should demonstrate:

INPUT → PROCESSING → STORAGE → SEARCH → CALCULATION → OUTPUT

using the C programming concepts covered so far.

The project should be functional, understandable and developed collaboratively.

Project A is about demonstrating what you have learned so far. It is not about building the most complicated system possible.